

# Questify — Project Report

## Questify — Comprehensive Academic Project Report

---

### Title Page

**Project Title:** Questify — A Retrieval-Augmented Generation (RAG) University Information Assistant

**Author:** [Student Name — Placeholder]

**Student ID:** [Roll No. — Placeholder]

**Department:** [Department of Computer Science & Engineering — Placeholder]

**Course / Programme:** [B.Tech. / M.Tech. Computer Science — Placeholder]

**Supervisor / Guide:** [Supervisor Name — Placeholder]

**Institution:** [University Name — Placeholder]

**Date of Submission:** 2026-04-05

---

### Abstract

Questify is a full-stack intelligent question-answering system designed for university environments. It combines a Python-based Retrieval-Augmented Generation (RAG) pipeline with a modern Next.js web frontend to let students and staff query institutional data — faculty directories, academic calendars, campus events, and subject resource playlists — in natural language. The backend indexes a SQLite relational database using the `intfloat/e5-small-v2` transformer embedding model and a FAISS (Facebook AI Similarity Search) vector store through the LlamaIndex framework. Retrieved document chunks are fed as grounding context to Google Gemini, which generates fluent, factual plain-English answers. The system exposes a FastAPI HTTP API and also ships a command-line interface (CLI). Authentication is handled by a lightweight SHA-256-hashed credential store. The frontend is a React 19/Next.js 15 single-page application styled with Tailwind CSS v4 and Radix UI primitives, providing a chat-style interaction model with conversation history persisted to `localStorage`. The repository covers database seeding scripts, indexing utilities, a log-file-based observability layer, and a component library built on `shadcn/ui` conventions.

---

# Keywords

Retrieval-Augmented Generation (RAG), FAISS, LlamaIndex, Sentence Embeddings, Google Gemini, FastAPI, Next.js, University Information System, Natural Language Query, SQLite, Vector Search, Conversational UI

---

## Table of Contents

1. Introduction
  2. Background and Literature Review
  3. System Design and Architecture
  4. Methodology and Implementation
  5. Results and Features
  6. Testing and Validation
  7. Performance and Security Considerations
  8. Limitations
  9. Future Scope
  10. Conclusion
  11. References
  12. Appendix A: Complete File List
  13. Appendix B: Language Composition
  14. Appendix C: Key Dependencies Summary
- 

## 1. Introduction

### 1.1 Problem Statement

University information — faculty contacts, academic calendars, examination schedules, departmental events, and learning resources — is typically scattered across multiple portals, PDFs, and notice boards. Students and staff face friction when they need quick, authoritative answers to questions such as: *“Who is the Head of Department of CSE?”*, *“When does the even semester begin?”*, or *“Where can I find a good DSA playlist?”*. Keyword-based search and static FAQ pages do not scale to the diversity of natural-language queries.

### 1.2 Objectives

1. Build a structured SQLite database capturing faculty, departments, events, academic calendar, subjects, and YouTube learning playlists for a university.
2. Construct a semantic vector index over that database using a lightweight transformer embedding model.
3. Implement a RAG pipeline that retrieves the most relevant database snippets for any user question and synthesises a plain-English answer via a large language model (Google Gemini).
4. Expose the pipeline via both a CLI and a REST API (FastAPI).

5. Deliver a polished, accessible, dark-mode-aware web frontend built with Next.js and Tailwind CSS.
  6. Provide user authentication to restrict access.
- 

## 2. Background and Literature Review

### 2.1 Retrieval-Augmented Generation (RAG)

RAG architectures couple a dense retrieval step with a generative model. Instead of relying solely on a language model's parametric knowledge, RAG first retrieves grounding passages from an external corpus and conditions the generator on those passages. This approach substantially reduces hallucination and is particularly well-suited to domain-specific corpora where up-to-date factual accuracy is critical — exactly the university information use case.

### 2.2 Vector Databases and FAISS

FAISS is an open-source library developed by Meta AI for efficient similarity search over dense floating-point vectors. It supports flat brute-force search as well as approximate nearest-neighbour methods such as IVF (Inverted File Index) and HNSW (Hierarchical Navigable Small World). In Questify, FAISS is accessed through the `llama-index-vector-stores-faiss` adapter, which wraps FAISS in LlamaIndex's storage context and provides transparent persistence to the `faiss_index/` directory.

### 2.3 Sentence Embeddings — E5 Family

The `intfloat/e5-small-v2` model belongs to Microsoft's E5 (EmbEddings from biDirEctional Encoder rEpresentations) family. It is a compact BERT-style model (~33 M parameters) tuned specifically for text embedding and semantic similarity tasks, making it practical for CPU-only deployments. Questify uses this model via the Hugging Face `transformers` library and performs mean pooling followed by L2 normalisation on the hidden states — a standard technique for producing fixed-dimensional sentence vectors.

### 2.4 LlamaIndex

LlamaIndex (formerly GPT Index) is a data framework for LLM applications. Its core abstractions — `Document`, `Node`, `VectorStoreIndex`, `QueryEngine` — are used extensively in Questify to pipeline document loading, chunking (via `SentenceSplitter`), embedding, vector indexing, and similarity-based retrieval.

### 2.5 Google Gemini API

Questify uses Google's `gemini-3-flash-preview` model (as seen in `gemini_engine.py:8`) through the `google-genai` Python package. The

model receives a structured prompt composed of the user question and retrieved context chunks, and returns a plain-English answer.

## 2.6 FastAPI and Modern Python Web Frameworks

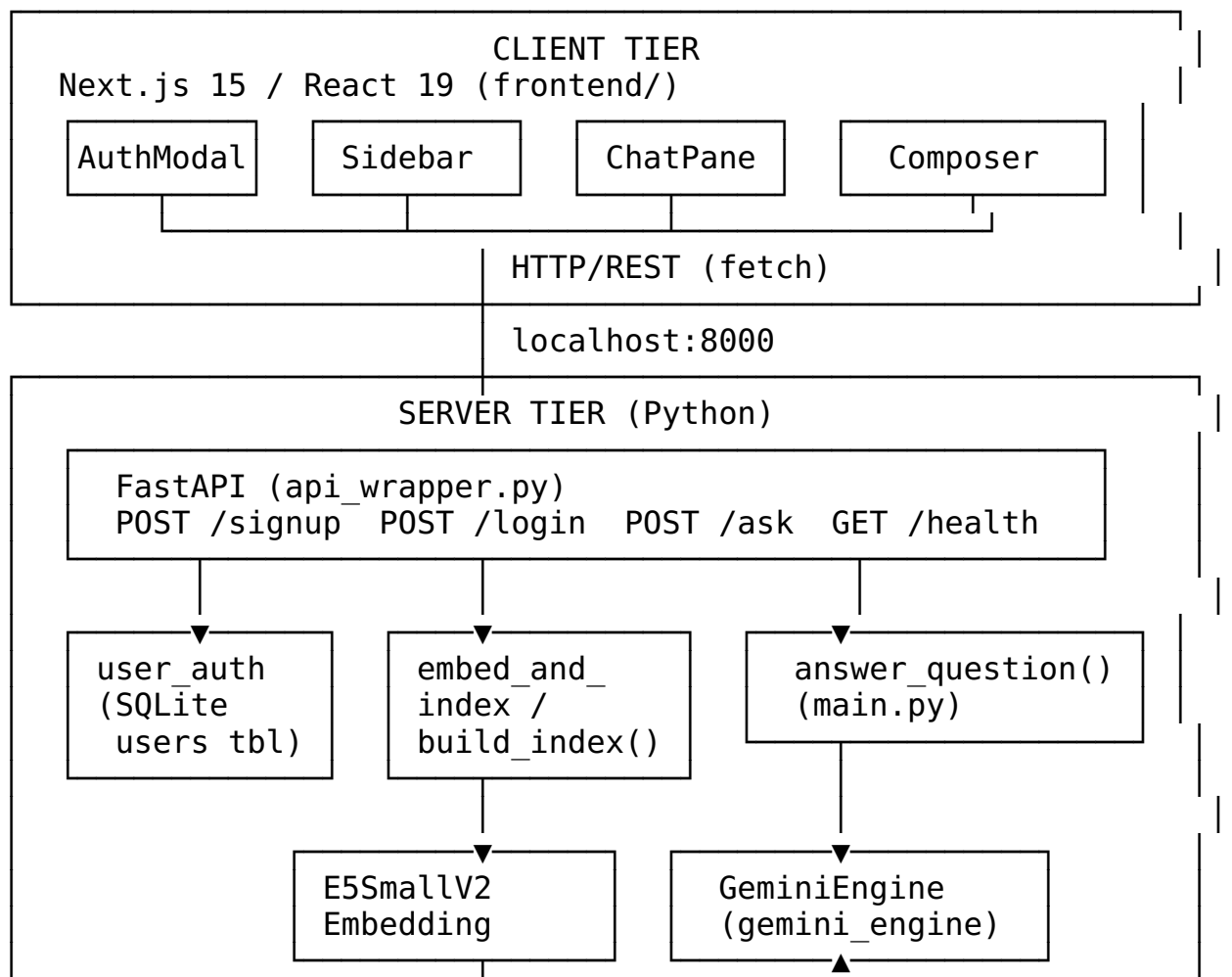
FastAPI is a high-performance Python ASGI web framework that generates automatic OpenAPI documentation and leverages Python type hints via Pydantic for request/response validation. Questify’s `api_wrapper.py` illustrates a minimal but complete FastAPI application pattern.

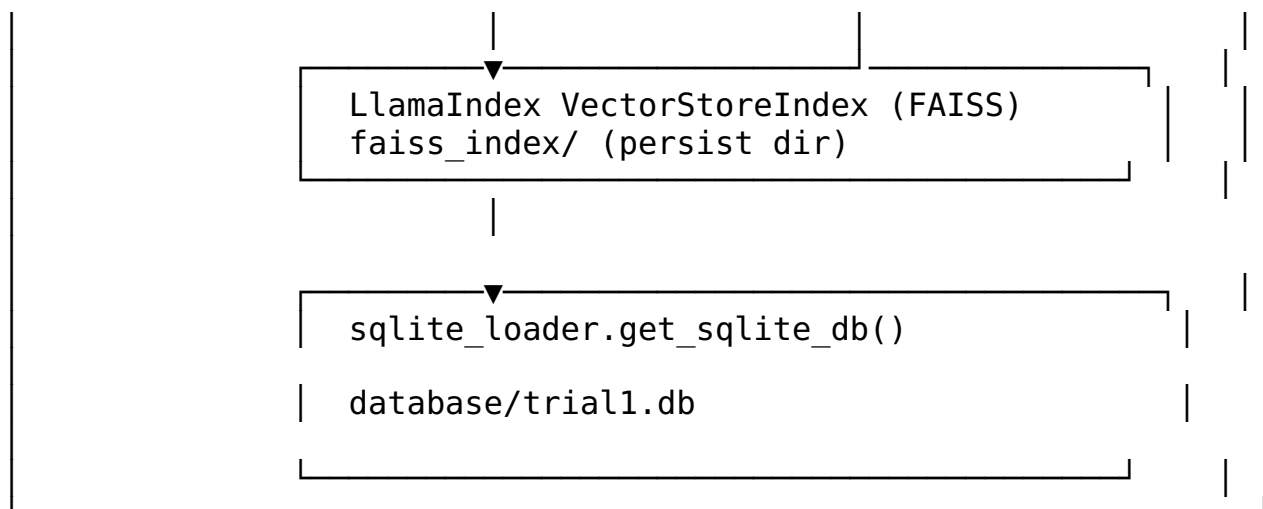
## 2.7 Next.js and React Ecosystem

Next.js 15 (with the App Router) provides server-side rendering, file-system routing, and a mature build pipeline for React applications. React 19 introduces improvements to concurrent rendering. The frontend leverages shadcn/ui-style component patterns (Radix UI primitives styled with Tailwind CSS v4), Framer Motion for animations, and Lucide React for iconography.

# 3. System Design and Architecture

## 3.1 High-Level Component Diagram





## 3.2 Data Flow

### Step 1 — Database Seeding (one-time)

Run `cse.py`, `department_faculty.py`, `events_academic_events.py`, `subjects_playlist.py` to populate `database/trial1.db` with 8 tables of university data.

### Step 2 — Indexing (`embed_and_index.py:build_index()`)

1. `sqlite_loader.get_sqlite_db()` reads every row from every user-defined table and converts each row to a `llama_index.core.schema.Document`. 2. `SentenceSplitter(chunk_size=1024, chunk_overlap=100)` splits documents into overlapping text nodes. 3. `E5SmallV2Embedding` computes a 384-dimensional L2-normalised embedding per node. 4. `VectorStoreIndex` inserts all embeddings into a FAISS flat index. 5. Index is persisted to `faiss_index/` via `LlamaIndex's StorageContext`.

**Step 3 — Query** (`main.answer_question()`) 1. Query engine retrieves top-20 candidates (`similarity_top_k=20`). 2. Scores are written to `logs/retrieval_debug.log`. 3. Most frequently occurring source table among retrieved nodes is inferred as the “most relevant table”. 4. Nodes with score  $\geq 0.5$  are kept as context (fallback: top-3). 5. Structured prompt sent to `GeminiEngine.generate()`. 6. Response is cleaned and returned.

### Step 4 — API Layer (`api_wrapper.py`)

Wraps the above in HTTP endpoints with CORS middleware, Pydantic request/response models, and structured logging.

### Step 5 — Frontend (`frontend/`)

A Next.js SPA makes fetch calls to the API; conversations and authentication state are persisted in `localStorage`.

---

## 4. Methodology and Implementation

### 4.1 Module 1: Database Seeding

#### `cse.py (root) / database/cse.py (mirror)`

- **Purpose:** Seeds CSE faculty data into `trial1.db`.
- **Tables created:** `department (id, name)`, `faculty (id, name, designation, department_id → FK to department)`.
- **Key operation (line 128):** Bulk inserts 82 faculty members for “Department of Computer Science & Engineering” using `cursor.executemany()`.
- **External deps:** `sqlite3 (stdlib)`.
- **Note:** Root-level and `database/` copies are identical in logic but connect to different relative working directory paths.

#### `department_faculty.py (root) / database/department_faculty.py (mirror)`

- **Purpose:** Seeds non-CSE departments (Economics, Journalism, Languages, Arts) and their faculty.
- **Tables:** `departments (department_id, department_name)`, `faculty (faculty_id, name, designation, department_id)`.
- **OOP pattern:** `Department` and `Faculty` classes each with `save()` / `get_id()` methods — an Active-Record style pattern.
- **Data volume:** 48 faculty entries across 4 departments.

#### `events_academic_events.py (root) / database/events_academic_events.py (mirror)`

- **Purpose:** Seeds campus events and academic calendar milestones.
- **Tables:** `events (id, title, description, event_date, location)`, `academic_calendar (id, title, start_date, end_date, description)`.
- **OOP pattern:** `Event` and `AcademicEvent` classes, each with `save_to_db()`.
- **Data volume:** 20 academic calendar entries (AY 2023–24) and 11 campus events (2023–2025).

#### `subjects_playlist.py (root) / database/subjects_playlist.py (mirror)`

- **Purpose:** Seeds CS subject names and YouTube playlist URLs.
- **Tables:** `subjects (subject_id, subject_name)`, `playlists (playlist_id, subject_id → FK, url)`.
- **Subjects seeded:** DAA, DSA, Operating System, Automata and Compiler Design, Java OOPs — 2 URLs each (10 URLs total).
- **OOP pattern:** `Subject` (with `IntegrityError` handling for duplicates) and `Playlist` classes.

## 4.2 Module 2: Configuration

### config.py

- **Purpose:** Centralises runtime paths and secrets.
- **Key constants:**
  - `MODEL_PATH` — hardcoded Windows path to a Mistral GGUF file (development artifact; not used in main flow since GeminiEngine is the active LLM).
  - `DB_PATH` = "database/trial1.db" (line 7).
  - `INDEX_PATH` = "faiss\_index" (line 8).
  - `SHIVAAY_API_KEY` = `os.getenv("SHIVAAY_API_KEY")` — read from .env; referenced but not actively consumed in the current codebase.
- **External deps:** python-dotenv.

## 4.3 Module 3: Authentication

### auth/user\_auth.py

- **Purpose:** Simple SQLite-backed user authentication.
- **Functions:**
  - `hash_password(password)` — SHA-256 hex digest (line 7-8).
  - `init_user_table()` — creates users table with `CREATE TABLE IF NOT EXISTS` (lines 10-21).
  - `signup(username, password)` — inserts new user; returns True on success, False on IntegrityError (lines 23-34).
  - `login(username, password)` — fetches stored hash and compares (lines 36-42).
- **Database:** database/trial1.db (hardcoded relative path, line 5).
- **Security note:** SHA-256 without salt is used. The README acknowledges this as intentionally simple; production systems should use bcrypt (already in requirements.txt) or Argon2.

## 4.4 Module 4: SQLite Loader

### sqlite\_loader.py

- **Purpose:** Converts a SQLite database into a list of LlamaIndex Document objects suitable for embedding.
- **Key function:** `get_sqlite_db(db_path: str) → list[Document]`
  - Discovers all non-system tables via `sqlite_master` (line 13).
  - For each row, constructs human-readable multi-line text: "Record from {table\_name} table:\nField: value\n..." (lines 25-49).
  - **FK join enrichment (lines 33-46):** Inspects `PRAGMA foreign_key_list` and for each FK column, fetches the referenced row and appends up to 2 columns of context. This enriches embedding context — e.g., a faculty row will include the linked department name.

- Each Document carries metadata={"table": table\_name} for downstream table-level inference.
- **External deps:** sqlite3 (stdlib), llama-index-core.

## 4.5 Module 5: Embedding and Indexing

### embed\_and\_index.py

- **Purpose:** Defines the custom embedding model and orchestrates index building/loading.

**Class E5SmallV2Embedding(BaseEmbedding)** (lines 15-48): - Subclasses LlamaIndex's BaseEmbedding. - `__init__`: Downloads/caches intfloat/e5-small-v2 tokenizer and model from HuggingFace; sets model to eval mode. - `_mean_pooling(model_output, attention_mask)`: Attention-mask-weighted mean of last hidden state token embeddings (lines 27-30). - `_get_embedding(text)`: Tokenizes, runs forward pass (no gradient), applies mean pooling, L2-normalises (lines 32-39). - Implements `_get_text_embedding`, `_get_query_embedding`, `_aget_query_embedding` as required by LlamaIndex.

**build\_index(db\_path, persist\_path)** (lines 51-66): - Orchestrates: `get_sqlite_db()` → `SentenceSplitter(1024, 100)` → `E5SmallV2Embedding()` → `VectorStoreIndex` → `persist`.

**load\_index(persist\_path)** (lines 68-71): - Loads persisted index from disk (available but unused in `api_wrapper.py` which always rebuilds).

## 4.6 Module 6: LLM Engine

### gemini\_engine.py

- **Purpose:** Thin wrapper around Google Generative AI.
- **Class GeminiEngine:**
  - `__init__`: Configures `google.generativeai` with `GEMINI_API_KEY` from environment; instantiates `gemini-3-flash-preview` (lines 6-8).
  - `generate(prompt: str) → str`: Calls `generate_content(prompt)` and returns `response.text.strip()` (lines 10-13).
- **External deps:** google-genai.

## 4.7 Module 7: Core RAG Logic and CLI

### main.py

- **Purpose:** Primary entrypoint — wires all modules together into a CLI chat assistant; also supplies utilities reused by the API.

**suppress\_output(to\_logfile=True)** (lines 25-37): Context manager redirecting stdout/stderr to `logs/llama.log` or `/dev/null` to silence noisy third-party logs.



**notify\_api(endpoint, data)** (lines 45–53): Best-effort POST to the locally running API server (CLI → API sync; errors logged but not fatal).

**answer\_question(index, question, model)** (lines 56–124): Core RAG function: 1. Creates query engine (similarity\_top\_k=20, similarity\_cutoff=0.3). 2. Executes vector search; logs all node scores to logs/retrieval\_debug.log. 3. Infers most relevant table via frequency count over metadata["table"]. 4. Filters nodes to score ≥ 0.5 (fallback: top-3). 5. Constructs structured prompt with rules (no SQL/JSON output, bullet lists for faculty queries). 6. Calls model.generate(prompt). 7. Post-processes for empty/trivial/negative responses.

**main() CLI loop** (lines 133–197): - Initialises DB, prompts login/signup. - Loads GeminiEngine, builds FAISS index. - Interactive REPL: reads user input, calls answer\_question, prints response. - **Note:** Lines 128–130 reference ShivaayEngine() in safe\_llm\_init() which is not imported or defined anywhere — dead code from a prior iteration.

## 4.8 Module 8: FastAPI HTTP Wrapper

### api\_wrapper.py

- **Purpose:** Wraps core logic in a REST API.
- **CORS middleware (lines 29–35):** allow\_origins=["\*"] — permissive for development.
- **Startup (lines 39–44):** init\_user\_table(), GeminiEngine(), build\_index() called at module load time.
- **Endpoints:**
  - GET /health → {"status": "ok"} (lines 56–58).
  - POST /signup — AuthRequest body → signup(); HTTP 400 on duplicate (lines 60–67).
  - POST /login — AuthRequest body → login(); HTTP 401 on failure (lines 69–76).
  - POST /ask — QuestionRequest body → answer\_question(); HTTP 500 on exception (lines 78–86).
- **Pydantic models:** AuthRequest(username, password), QuestionRequest(question) (lines 47–51).
- **External deps:** fastapi, uvicorn, pydantic.

## 4.9 Module 9: Frontend

### frontend/app/layout.tsx

- **Purpose:** Next.js root layout — sets <title>AskQuery, loads Google Fonts (Geist, Geist Mono), includes Vercel Analytics, applies global CSS.

### frontend/app/page.tsx

- **Purpose:** The sole Next.js page; renders <AIAssistantUI /> at the / route (4 lines).

## frontend/components/AIAssistantUI.jsx

- **Purpose:** Root client component and application shell; orchestrates all state.
- **State managed:** Authentication, theme, backend health status, conversations (localStorage-persisted), sidebar collapse, “thinking” indicator.
- **Key methods:**
  - `createNewChat()` — generates random ID, prepends new conversation object.
  - `sendMessage(convId, content)` — appends user message, calls backend `/ask`, appends assistant reply.
  - `editMessage / resendMessage` — mutate conversation history.
  - `togglePin(id)` — flips the pinned flag.
- **Keyboard shortcuts:** Ctrl/Cmd+N → new chat; / → focus search; Escape → close sidebar.

## frontend/components/AuthModal.jsx

- **Purpose:** Modal dialog for login/signup toggle.
- **On login success:** Saves username to `localStorage`; calls `onAuthSuccess`.
- **Error UX:** Distinguishes backend connection errors from credential failures.

## frontend/components/ChatPane.jsx

- **Purpose:** Renders the chat message list with in-place editing and thinking indicator.
- **ThinkingMessage** (local component): Animated bouncing dots with a “Pause” button.
- **Features:** Message editing (Save / Save & Resend / Cancel), resend, relative timestamps.

## frontend/components/Composer.jsx

- **Purpose:** Auto-expanding textarea. Grows up to 12 lines, then scrolls. Enter sends; Shift+Enter adds newline.

## frontend/components/Sidebar.jsx

- **Purpose:** Collapsible navigation sidebar with pinned and recent conversations.
- **Animation:** Framer Motion spring collapse/expand (width: 320 → 64).
- **Responsive:** Full overlay drawer on mobile; static side panel on  $\geq$  md breakpoint.
- **Integrates:** `SearchModal` for full-screen search overlay.

## **frontend/components/Header.jsx**

- **Purpose:** Sticky top bar with AI Assistant branding and mobile sidebar toggle.

## **frontend/components/Message.jsx**

- **Purpose:** Renders a single chat bubble — user messages right-aligned (dark), assistant left-aligned (light border). Avatars: “JD” (user), “AI” (assistant).

## **frontend/components/ThemeToggle.jsx**

- **Purpose:** Sun/Moon toggle button; toggles between "light" and "dark".

## **frontend/components/SearchModal.jsx**

- **Purpose:** Full-screen search overlay grouping conversations into Today / Yesterday / Previous 7 Days / Older time buckets.

## **frontend/components/ConversationRow.jsx**

- **Purpose:** Sidebar conversation item with title, relative timestamp, message count, and hover-reveal pin/unpin star button. Desktop hover shows a preview tooltip.

## **frontend/components/SidebarSection.jsx**

- **Purpose:** Collapsible section wrapper with Framer Motion height animation.

## **frontend/components/utils.js**

- **Purpose:** Shared utilities.
  - `cls(...c)`: Joins truthy class names.
  - `timeAgo(date)`: Human-readable relative time via `Intl.RelativeTimeFormat`.
  - `makeId(prefix)`: Short random ID generator.

## **frontend/components/mockData.js**

- **Purpose:** Static fixture data (`INITIAL_CONVERSATIONS`, `INITIAL_TEMPLATES`, `INITIAL_FOLDERS`) for development/UI prototyping. Not wired into live application state.

## **frontend/components/theme-provider.tsx**

- **Purpose:** Re-exports next-themes ThemeProvider with TypeScript typing. Available but theme is managed manually via classList in AIAssistantUI.jsx.

## **frontend/app/globals.css and frontend/styles/globals.css**

- **Purpose:** Identical Tailwind CSS v4 global stylesheets (duplicated between app/ and styles/).
- Define design tokens (--background, --foreground, --primary, etc.) in both :root (light) and .dark using the oklch colour space.
- Register Geist/Geist Mono as CSS custom font properties.

## **frontend/components/ui/ — Radix UI / shadcn-style Component Library**

~40 TypeScript files wrapping Radix UI headless components with Tailwind utility styling. Files include: accordion, alert-dialog, alert, aspect-ratio, avatar, badge, breadcrumb, button-group, button, calendar, card, carousel, chart, checkbox, collapsible, command, context-menu, dialog, drawer, dropdown-menu, empty, field, form, hover-card, input-group, input-otp, input, item, kbd, label, menubar, navigation-menu, pagination, popover, progress, radio-group, resizable, scroll-area, select, separator, sheet, sidebar, skeleton, slider, sonner, spinner, switch, table, tabs, textarea, toast, toaster, toggle-group, toggle, tooltip, use-mobile, use-toast. Most are available as a reusable library but not yet consumed by current application components.

## **frontend/hooks/use-mobile.ts and frontend/hooks/use-toast.ts**

- **Purpose:** Custom React hooks for responsive breakpoint detection and toast notification management (shadow the identical files in components/ui/).

---

# **5. Results and Features**

## **5.1 Implemented and Operational Features**

| <b>Feature</b>       | <b>Implementation File</b>         |
|----------------------|------------------------------------|
| User signup          | auth/user_auth.py → POST /signup   |
| User login           | auth/user_auth.py → POST /login    |
| Health check         | api_wrapper.py → GET /health       |
| Natural language Q&A | main.answer_question() → POST /ask |
| CLI interactive chat | main.py                            |
| FAISS index build    | embed_and_index.build_index()      |

| Feature                                      | Implementation File                       |
|----------------------------------------------|-------------------------------------------|
| Index persistence                            | faiss_index/ directory                    |
| Retrieval debug logging                      | logs/retrieval_debug.log                  |
| System event logging                         | logs/system.log                           |
| Web UI — auth flow                           | AuthModal.jsx                             |
| Web UI — chat interface                      | ChatPane.jsx + Composer.jsx               |
| Web UI — conversation history (localStorage) | AIAssistantUI.jsx                         |
| Web UI — pin/unpin conversations             | AIAssistantUI.jsx + ConversationRow.jsx   |
| Web UI — conversation search                 | SearchModal.jsx                           |
| Web UI — dark/light mode                     | ThemeToggle.jsx + globals.css             |
| Web UI — animated sidebar                    | Sidebar.jsx (Framer Motion)               |
| Web UI — backend status indicator            | AIAssistantUI.jsx health check with retry |
| Web UI — message editing                     | ChatPane.jsx                              |

## 5.2 UI / API Flows

**Login Flow:** 1. On load, AIAssistantUI checks localStorage for username. 2. If absent, AuthModal is shown (blocking). 3. User submits credentials → POST /login → 200 → username stored → chat rendered.

**Q&A Flow:** 1. User types question in Composer; presses Enter. 2. sendMessage() appends user message to conversation state. 3. POST /ask sent to backend; “thinking” animation shown. 4. Response appended as assistant message; conversation persisted to localStorage.

**Index Build Flow (server startup):** build\_index() → get\_sqlite\_db() → documents → SentenceSplitter → nodes → E5SmallV2Embedding → FAISS → persist to faiss\_index/.

## 5.3 Database Content (as seeded)

The database/trial1.db contains 8 tables:

| Table             | Rows                                       | Source Script             |
|-------------------|--------------------------------------------|---------------------------|
| users             | Runtime-populated                          | auth/user_auth.py         |
| department        | 1 (CSE)                                    | cse.py                    |
| faculty (CSE)     | 82                                         | cse.py                    |
| departments       | 4 (Economics, Journalism, Languages, Arts) | department_faculty.py     |
| faculty (non-CSE) | 48                                         | department_faculty.py     |
| events            | 11                                         | events_academic_events.py |
| academic_calendar | 20                                         | events_academic_events.py |
| subjects          | 5                                          | subjects_playlist.py      |

| Table     | Rows | Source Script        |
|-----------|------|----------------------|
| playlists | 10   | subjects_playlist.py |

---

## 6. Testing and Validation

### 6.1 Existing Tests

**No formal automated test suite is present in the repository.** There are no tests/ directory, no pytest configuration files, and no test files in the repository tree.

### 6.2 Proposed Validation Approach

**Unit Tests (Python — pytest):** - test\_hash\_password: assert hash\_password("pw") == hashlib.sha256(b"pw").hexdigest(). - test\_signup\_login: mock SQLite; assert signup() returns True for new user, False for duplicate; login() returns correct boolean. - test\_get\_sqlite\_db: use an in-memory SQLite DB with known rows; assert correct Document count and text formatting. - test\_e5\_embedding\_shape: assert get\_embedding("hello") returns shape (384,) and is unit-normalised (L2 norm  $\approx 1.0$ ). - test\_gemini\_engine\_generate: mock genai.GenerativeModel.generate\_content; assert output equals response.text.strip().

**Integration Tests:** - Build index against a fixture DB; run answer\_question() with known questions; assert answer contains expected keywords. - Start FastAPI app with TestClient (httpx); test all four endpoints end-to-end with mocked GeminiEngine.

**Frontend Tests (Jest / React Testing Library):** - Render AuthModal with isOpen=true; simulate form submission; assert API called. - Render ChatPane with sample messages; assert all messages rendered. - Test timeAgo() utility with fixed dates.

**RAG Quality Evaluation:** - Construct a golden dataset of 20–30 question/expected-answer pairs covering each table domain. - Compute exact-match recall and keyword-hit metrics. - Track retrieval scores to tune similarity\_top\_k and the 0.5 score cutoff.

---

## 7. Performance and Security Considerations

### 7.1 Performance

| Concern                        | Detail                                                                                                               |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Index rebuild on every startup | api_wrapper.py:44 always calls build_index(); load_index() exists but is unused. Large databases cause slow startup. |

| Concern                   | Detail                                                                                                                   |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Embedding model load time | intfloat/e5-small-v2 downloads from HuggingFace on first run; subsequent runs use the local cache.                       |
| Retrieval depth           | similarity_top_k=20 retrieves more candidates than necessary; reducing to 5-10 would lower LLM context size and latency. |
| Frontend bundle size      | ~40 unused ui/ components; tree-shaking via Next.js should eliminate them but should be verified.                        |

## 7.2 Security

| Concern                                      | Severity                        | Notes                                                                                                                                                                                                                                              |
|----------------------------------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SHA-256 without salt for passwords           | <b>High</b>                     | auth/user_auth.py:8.<br>No salt makes the system vulnerable to rainbow table attacks. Use bcrypt (already in requirements.txt) or Argon2.                                                                                                          |
| CORS wildcard                                | <b>Medium</b>                   | api_wrapper.py:30.<br>allow_origins=["*"] should be restricted to the known frontend origin in production.<br>Any unauthenticated caller can invoke the Gemini API, potentially incurring costs. A session token or JWT should gate this endpoint. |
| No authentication on /ask endpoint           | <b>Medium</b>                   | AuthModal.jsx:6 and AIAssistantUI.jsx:9 import apiService from "../lib/api". This file does not exist in the repository; the frontend cannot build or run without it.                                                                              |
| Missing frontend/lib/api.js                  | <b>Critical (functionality)</b> | MODEL_PATH at line 6 is a developer machine path; not a security risk but breaks portability.                                                                                                                                                      |
| Hardcoded Windows path in config.py          | <b>Low</b>                      | Table names from sqlite_master are injected directly; low                                                                                                                                                                                          |
| SQL string interpolation in sqlite_loader.py | <b>Low</b>                      |                                                                                                                                                                                                                                                    |

| Concern                   | Severity | Notes                                                                                                          |
|---------------------------|----------|----------------------------------------------------------------------------------------------------------------|
| .env correctly gitignored | N/A      | risk since the source is trusted, but not parameterised.<br>.gitignore:5 excludes .env*; no secrets committed. |

---

## 8. Limitations

1. **Missing frontend/lib/api.js:** The apiService import is referenced in AuthModal.jsx and AIAssistantUI.jsx but the file is absent from the repository. The frontend cannot function without this API client module.
  2. **Index rebuilt on every startup:** api\_wrapper.py always calls build\_index(), discarding any persisted index. For a database of significant size, this is prohibitively slow.
  3. **Unsalted SHA-256 passwords:** The authentication scheme does not use a modern password hashing algorithm.
  4. **Dead code — ShivaayEngine:** main.py:128–130 references ShivaayEngine(), which is not imported, defined, or present anywhere in the repository.
  5. **Duplicate seeding scripts:** Root-level seeding scripts are identical to their counterparts in database/, creating maintenance confusion.
  6. **No server-side session validation:** User sessions in the frontend rely purely on localStorage username storage with no server-side session token.
  7. **No error recovery on index failure:** If build\_index() fails at startup (e.g., DB not seeded), the API server crashes without a helpful error message.
  8. **mockData.js not wired:** INITIAL\_CONVERSATIONS, INITIAL\_TEMPLATES, and INITIAL\_FOLDERS are defined but never imported or used in the live application.
  9. **Hardcoded model name:** gemini\_engine.py:8 hardcodes "gemini-3-flash-preview". A configurable model name would be more robust.
  10. **No pagination for large databases:** sqlite\_loader.get\_sqlite\_db() loads all rows with cursor.fetchall(), which could exhaust memory on large databases.
-



## 9. Future Scope

1. **Implement frontend/lib/api.js** — A minimal fetch-based API client to fix the broken frontend build.
  2. **Incremental index updates** — Track DB row checksums or timestamps; only re-embed changed rows rather than rebuilding the full index on every startup.
  3. **Session-based authentication** — Replace localStorage username with JWT-based sessions; validate token on the /ask endpoint.
  4. **Upgrade password hashing** — Use bcrypt (already in requirements.txt but unused) or Argon2.
  5. **Streaming responses** — Use FastAPI StreamingResponse with Gemini streaming output to improve perceived latency.
  6. **Multi-university support** — Parameterise DB schema and seeding for multiple institutions; add a tenant ID to documents.
  7. **Feedback and RAG evaluation loop** — Add thumbs-up/thumbs-down UI; log feedback to DB; use to fine-tune retrieval thresholds and prompt templates.
  8. **Expand the knowledge base** — Integrate timetables, room allocations, fee structures, hostel information.
  9. **Production deployment** — Dockerise the backend; deploy Next.js to Vercel (Analytics integration already present); use PostgreSQL with pgvector in place of FAISS files.
  10. **Automated test suite** — Introduce pytest for backend and Jest/RTL for frontend; add CI/CD with GitHub Actions.
  11. **Accessibility audit** — Review all components against WCAG 2.1 AA; Radix UI primitives provide keyboard navigation but labels/ARIA roles need verification.
- 

## 10. Conclusion

Questify demonstrates a practical, end-to-end implementation of the Retrieval-Augmented Generation paradigm applied to the university information access domain. It successfully integrates a relational SQLite database, a transformer-based embedding model (intfloat/e5-small-v2), a FAISS vector store via LlamaIndex, and a Google Gemini language model into a coherent pipeline. The Python backend, exposed through both a CLI and a FastAPI REST API, is complemented by a polished React/Next.js web frontend with dark-mode support, conversation history, and an animated sidebar.

The system addresses a real information-access problem — the difficulty of querying fragmented university data using natural language — and provides a reasonable prototype solution. Key limitations (missing `lib/api` module, lack of tests, unsalted password hashing, full index rebuild at startup) represent clear, actionable items for a production-ready iteration. The modular architecture — with clear separation between seeding, loading, embedding, retrieval, generation, API, and UI layers — provides a solid foundation for future improvements.

---

## 11. References

*No formal references are cited within the repository codebase or README. The following are real packages and technologies identified from `requirements.txt` and `frontend/package.json`; no fabricated citations are included.*

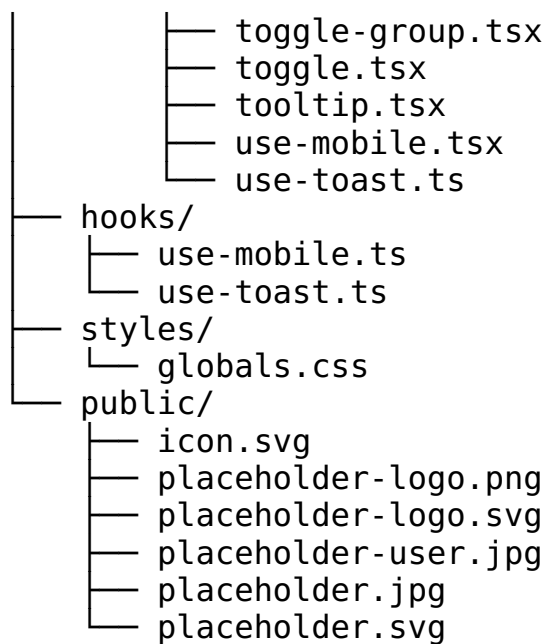
- `intfloat/e5-small-v2` — HuggingFace model card: <https://huggingface.co/intfloat/e5-small-v2>
  - LlamaIndex documentation: <https://docs.llamaindex.ai>
  - FAISS (Facebook AI Similarity Search): <https://github.com/facebookresearch/faiss>
  - FastAPI documentation: <https://fastapi.tiangolo.com>
  - Google Generative AI Python SDK: <https://pypi.org/project/google-genai/>
  - Next.js 15: <https://nextjs.org/docs>
  - Radix UI: <https://www.radix-ui.com>
  - Tailwind CSS v4: <https://tailwindcss.com>
  - Framer Motion: <https://www.framer.com/motion/>
- 

## Appendix A: Complete File List

```
questify/
├── .gitignore
├── README.md
├── requirements.txt
├── config.py
├── main.py
├── api_wrapper.py
├── gemini_engine.py
├── embed_and_index.py
├── sqlite_loader.py
├── cse.py
├── department_faculty.py
├── events_academic_events.py
├── subjects_playlist.py
├── trial1.db
├── PROJECT_REPORT.md           ← this file
├── auth/
│   └── user_auth.py
```

- database/
  - cse.py
  - department\_faculty.py
  - events\_academic\_events.py
  - subjects\_playlist.py
  - trial1.db
- faiss\_index/
  - default\_vector\_store.json
  - docstore.json
  - graph\_store.json
  - image\_vector\_store.json
  - index\_store.json
- logs/
  - llama.log
  - retrieval\_debug.log
  - system.log
- frontend/
  - .gitignore
  - components.json
  - next.config.mjs
  - package.json
  - package-lock.json
  - pnpm-lock.yaml
  - postcss.config.mjs
  - tsconfig.json
  - app/
    - globals.css
    - layout.tsx
    - page.tsx
  - components/
    - AIAssistantUI.jsx
    - AuthModal.jsx
    - ChatPane.jsx
    - Composer.jsx
    - ComposerActionsPopover.jsx
    - ConversationRow.jsx
    - CreateFolderModal.jsx
    - CreateTemplateModal.jsx
    - FolderRow.jsx
    - GhostIconButton.jsx
    - Header.jsx
    - Message.jsx
    - SearchModal.jsx
    - SettingsPopover.jsx
    - Sidebar.jsx
    - SidebarSection.jsx
    - TemplateRow.jsx
    - ThemeToggle.jsx
    - mockData.js
    - theme-provider.tsx
    - utils.js
    - ui/

- accordion.tsx
- alert-dialog.tsx
- alert.tsx
- aspect-ratio.tsx
- avatar.tsx
- badge.tsx
- breadcrumb.tsx
- button-group.tsx
- button.tsx
- calendar.tsx
- card.tsx
- carousel.tsx
- chart.tsx
- checkbox.tsx
- collapsible.tsx
- command.tsx
- context-menu.tsx
- dialog.tsx
- drawer.tsx
- dropdown-menu.tsx
- empty.tsx
- field.tsx
- form.tsx
- hover-card.tsx
- input-group.tsx
- input-otp.tsx
- input.tsx
- item.tsx
- kbd.tsx
- label.tsx
- menubar.tsx
- navigation-menu.tsx
- pagination.tsx
- popover.tsx
- progress.tsx
- radio-group.tsx
- resizable.tsx
- scroll-area.tsx
- select.tsx
- separator.tsx
- sheet.tsx
- sidebar.tsx
- skeleton.tsx
- slider.tsx
- sonner.tsx
- spinner.tsx
- switch.tsx
- table.tsx
- tabs.tsx
- textarea.tsx
- toast.tsx
- toaster.tsx



---

## Appendix B: Language Composition

| Language   | Percentage | Primary Files                                                                                                                  |
|------------|------------|--------------------------------------------------------------------------------------------------------------------------------|
| TypeScript | 54.7%      | frontend/components/ui/*.tsx, frontend/app/layout.tsx, frontend/hooks/*.ts, frontend/components/theme-provider.tsx             |
| JavaScript | 24.9%      | frontend/components/*.jsx, frontend/components/utills.js, frontend/components/mockData.js, frontend/next.config.mjs            |
| Python     | 17.9%      | main.py, api_wrapper.py, gemini_engine.py, embed_and_index.py, sqlite_loader.py, config.py, auth/user_auth.py, seeding scripts |
| CSS        | 2.5%       | frontend/app/globals.css, frontend/styles/globals.css                                                                          |

---

# Appendix C: Key Dependencies Summary

## Python Backend (requirements.txt)

| Package                         | Role                                                                     |
|---------------------------------|--------------------------------------------------------------------------|
| fastapi                         | REST API framework                                                       |
| uvicorn                         | ASGI server                                                              |
| bcrypt                          | Password hashing (in requirements; not yet used in code)                 |
| requests                        | HTTP client (CLI → API sync in main.py)                                  |
| numpy                           | Numerical arrays for embeddings                                          |
| faiss-cpu                       | Vector similarity search                                                 |
| sentence-transformers           | Embedding library (in requirements; E5 loaded directly via transformers) |
| llama-index-core                | RAG framework core abstractions                                          |
| llama-index-vector-stores-faiss | FAISS adapter for LlamaIndex                                             |
| llama-index-embeddings-openai   | LlamaIndex embedding base class (parent of E5SmallV2Embedding)           |
| openai                          | Transitive dependency                                                    |
| python-dotenv                   | .env file loading                                                        |
| google-generativeai             | Google Gemini API client                                                 |

## Frontend (frontend/package.json — selected)

| Package           | Version  | Role                              |
|-------------------|----------|-----------------------------------|
| next              | 15.2.4   | React meta-framework              |
| react             | ^19      | UI library                        |
| tailwindcss       | ^4.1.9   | Utility-first CSS framework       |
| @radix-ui/*       | various  | Headless accessible UI primitives |
| framer-motion     | latest   | Animation library                 |
| lucide-react      | ^0.454.0 | Icon library                      |
| next-themes       | ^0.4.6   | Theme management                  |
| @vercel/analytics | 1.3.1    | Usage analytics                   |
| zod               | 3.25.76  | Schema validation                 |
| react-hook-form   | ^7.60.0  | Form state management             |
| cmdk              | 1.0.4    | Command-palette component         |
| sonner            | ^1.7.4   | Toast notifications               |
| typescript        | ^5       | Static type checking              |

---

*End of Report*